ADR_007 模組間事件採用 Outbox

> 狀態：已接受
> 日期：2026-10-14

背景

模組之間以事件溝通（《設計_001_領域模型》第 7 節）。例如「付款已成功」需要讓訂單、帳本、報表各自反應。其中帳本與訂單涉及金額與庫存，事件絕不能遺失；而帳本記帳失敗也不應該擋住付款的處理。

考慮過的選項

選項	優點	缺點
同步，在同一個交易內處理	最簡單；事件不會遺失	任一訂閱者失敗就撤銷整件事，帳本會擋住業務；交易變長；日後接上訊息佇列時行為完全改變
交易提交後非同步處理	解耦、快速	提交後、處理前當機，或訂閱者失敗，事件就遺失，也不會重試
Outbox	事件與業務資料在同一交易內寫入，不會遺失；訂閱者失敗會重試；日後只需把發送目的地改為訊息佇列	同一事件可能被處理兩次；有數秒以內的延遲；多一張資料表與發送機制

實作方式另有兩個選項：自行實作，或使用 Spring Modulith 的事件持久化機制。後者已實現上述行為，並能記錄每個訂閱者各自的處理狀態。

決定

- 採用 Outbox，以 Spring Modulith 的事件持久化機制實現
- 所有事件訂閱者都必須冪等，以事件編號判斷是否處理過
- 未完成的事件須定期重新發送，不只在應用程式重新啟動時
- 撰寫測試驗證機制本身的行為：訂閱者失敗後會被重送、重送只送給失敗的訂閱者、重複收到同一事件不會重複處理
- 結帳流程中的模組呼叫仍為同步呼叫，不經過事件（見 ADR_005）

後果

- 好處：金額與庫存相關的事件不會遺失；模組之間互不阻擋；日後接上訊息佇列的改動範圍很小
- 代價：每個訂閱者都要處理重複事件；事件延遲造成「其他模組的狀態暫時是舊的」，所有讀取這類狀態的地方都要考慮事件尚未送達的情況
- 重新檢討的條件：引入訊息佇列時，新增 ADR 說明事件如何轉送到佇列

附錄：事件尚未送達時的檢查清單

以下是已知「可能讀到舊狀態」的地方與對策。實作與審查程式時逐項對照；之後發現新的情況時補充到這裡。

情境	風險	對策	出處
逾時取消	付款已成功，但結帳單仍顯示待付款，被誤取消	取消前向付款模組確認沒有成功或處理中的付款	設計_002 3.7
會員主動取消	同上	同上	設計_002 3.7
每日結算	當天完成的子訂單事件還沒送到結算模組，被漏掉	結算前確認「子訂單已完成」事件都已處理	設計_002 3.5
每日報表	報表依賴的事件還沒處理完，數字不完整	產生報表前確認相關事件都已處理	設計_002 3.5
賣家查詢新訂單	付款剛成功，賣家還看不到子訂單	需求接受 5 秒內出現	需求_001 3.11
結帳後的購物車	結帳後立刻回到購物車，仍短暫看到已結帳的商品	接受數秒延遲	設計_001 第 7 節
會員查詢付款結果	訂單尚未更新為已付款	付款結果直接由付款模組回答，不經過訂單	設計_002 3.2